Software Testing - Project 2

Formal Criteria-Based Test Design & Automation

Premium Discount Eligibility System

Team Members:

Aya Alkhateeb 2230887 - No.12

Alaa Alnajjar 2232195 - No.15

Saba Ramadeen 2038075 - No.1

Introduction

This report presents the formal test design and automated testing approach for the Premium Discount Eligibility System. The system calculates a customer discount percentage based on customer type, number of orders in the last year, and newsletter subscription status. The test suite was designed using Model-Driven Test Design and Input Space Partitioning, then automated using JUnit 5 parameterized tests.

Testable Function

The testable function is DiscountCalculator.calculateDiscount(customerType, totalOrdersInLastYear, isSubscribedToNewsletter). The function returns an integer discount percentage for valid inputs and throws IllegalArgumentException for invalid inputs or business-constraint violations.

Functional Rules Reflected in the Code

- All customers receive a 5% base discount.
- If isSubscribedToNewsletter is true, 2% is added.
- NEW customers receive +0%, REGULAR customers receive +3%, and PREMIUM customers receive +5%.
- If totalOrdersInLastYear is 10 or more, 5% is added as a loyalty bonus.
- A NEW customer with 10 or more orders violates the stated business constraint and must throw IllegalArgumentException.
- The final discount is capped at 15%.
- Invalid customer types and negative order counts are rejected with IllegalArgumentException.

Input Characteristics

The input space is modeled using three characteristics taken directly from the system requirements:

- customerType
- totalOrdersInLastYear
- isSubscribedToNewsletter

Input Space Partitioning Blocks

Characteristic	Block ID	Block	Representative Value
customerType	C1	NEW	"NEW"
	C2	REGULAR	"REGULAR"
	C3	PREMIUM	"PREMIUM"
	C4	Invalid customer type	"VIP"
totalOrdersInLastYear	O1	0-9 orders	0 or 5
	O2	10 or more orders	10
	O3	Negative orders	-1
isSubscribedToNewsletter	N1	Not subscribed	false
	N2	Subscribed	true

Completeness and Disjointness Justification

customerType: The blocks are disjoint because a customer type is either one of the valid defined types or an invalid unsupported type. They are complete for the considered input domain because they include all valid customer types required by the system plus an invalid customer type block for rejected values.

totalOrdersInLastYear: The blocks are disjoint because an order count is either negative, between 0 and 9, or 10 or more. They are complete for the integer input domain considered by this method because any integer order count falls into exactly one of these three blocks.

isSubscribedToNewsletter: The blocks are disjoint and complete because the value is Boolean. A customer is either subscribed or not subscribed.

Pair-Wise Coverage Design

Pair-Wise Coverage requires every block from each characteristic to be combined with every block of every other characteristic at least once. Since the largest two characteristics have 4 and 3 blocks, a Pair-Wise test suite for this model requires at least 12 abstract combinations in principle. The selected 12 test cases provide a compact Pair-Wise design for the defined blocks and align with the uploaded JUnit parameterized tests.

Pair-Wise Test Cases

TC	Customer Block	Orders Block	Newsletter Block	Concrete Input	Expected Result
TC1	C1	O1	N1	NEW, 0, false	5
TC2	C1	O2	N2	NEW, 10, true	IllegalArgumentException
TC3	C1	O3	N1	NEW, -1, false	IllegalArgumentException
TC4	C2	O1	N2	REGULAR, 0, true	10
TC5	C2	O2	N1	REGULAR, 10, false	13
TC6	C2	O3	N2	REGULAR, -1, true	IllegalArgumentException
TC7	C3	O1	N1	PREMIUM, 0, false	10
TC8	C3	O2	N2	PREMIUM, 10, true	15
TC9	C3	O3	N1	PREMIUM, -1, false	IllegalArgumentException
TC10	C4	O1	N2	VIP, 5, true	IllegalArgumentException
TC11	C4	O2	N1	VIP, 10, false	IllegalArgumentException
TC12	C4	O3	N2	VIP, -1, true	IllegalArgumentException

Pair Coverage Check

The selected test cases satisfy Pair-Wise Coverage as follows:

- **customerType x totalOrdersInLastYear:** Each customer type block is paired with each order-count block. NEW, REGULAR, PREMIUM, and the invalid customer type block all appear with 0-9, 10+, and negative order blocks.
- **customerType x isSubscribedToNewsletter:** Each customer type block appears with both true and false newsletter values across the table.
- **totalOrdersInLastYear x isSubscribedToNewsletter:** Each order-count block appears with both true and false newsletter values across the table.

Infeasible Combination

The pair customerType = NEW and totalOrdersInLastYear = 10 or more is infeasible according to the system requirements. A NEW customer cannot mathematically have 10 or more orders in the last year. Therefore, TC2 is expected to throw IllegalArgumentException instead of returning a discount value.

Invalid inputs and infeasible combinations are treated differently in this model. Invalid inputs are rejected because the individual value itself is not acceptable, such as an unsupported customer type or a negative order count. The infeasible combination is customerType = NEW with totalOrdersInLastYear = 10 or more, because these two blocks cannot logically occur together according to the business constraint.

Automated Testing Approach

The test suite was implemented using JUnit 5 parameterized tests. The valid discount cases are executed through one @ParameterizedTest method using @CsvSource. This method checks the expected integer discount values using assertEquals. The invalid input and infeasible constraint cases are executed through another @ParameterizedTest method using @CsvSource and assertThrows to verify that IllegalArgumentException is thrown.

The uploaded test code contains two parameterized test methods: testValidDiscounts for valid discount values and testInvalidInputs for invalid or constraint-violating inputs. This structure keeps the tests concise while still mapping each row to the Pair-Wise design table.

Limitations of Pair-Wise Testing

Although Pair-Wise Testing is effective for reducing the number of test cases, achieving 100% Pair-Wise Coverage does not guarantee that the system is bug-free. Pair-Wise Testing focuses only on interactions between pairs of input characteristics. As a result, it may miss faults that appear only when three or more input values interact together.

For example, a defect might occur only when the customer type is PREMIUM, the customer is subscribed to the newsletter, and the number of orders is 10 or more. If such a defect depends on the three values together, Pair-Wise Coverage may not always reveal it. Therefore, Pair-Wise Testing is useful for systematic and efficient test design, but it is not a complete replacement for exhaustive testing, boundary value analysis, or additional domain-specific test cases.

Team Contribution Matrix

Team Member	Contribution	Related File(s)	Commit Evidence
Aya Alkhateeb	Designed the Input Space Partitioning model, derived the unified Pair-Wise Coverage table, and explained the infeasible combination.	ProjectReport.pdf / ProjectReport.docx	Add ISP and pair-wise test design
Alaa Alnajjar	Implemented the DiscountCalculator logic according to the functional discount rules, validation rules, and maximum cap.	DiscountCalculator.java	implementation DiscountCalculator
Saba Ramadeen	Implemented the JUnit 5 parameterized tests using @ParameterizedTest and @CsvSource for valid, invalid, and infeasible cases.	DiscountCalculatorTest.java	Add unit tests for DiscountCalculator
All Team Members	Reviewed the final report, verified the required files, and ensured the repository includes the implementation, tests, and PDF report.	All files	Add final project report